

2 观察者模式

让你的对象 保持消息灵通



嘿，Jerry，我正在通知大家，模式小组会议改到周六晚上。我们打算讨论观察者模式。这个模式最棒了！
Jerry，最棒的！

你不想错过有趣的事情，对吧？有一个模式可以让你的对象在发生某些事情时保持消息灵通。它就是观察者模式。它是最常用的设计模式之一，而且是不可思议的好用。我们打算介绍观察者的各种有趣的方面，像一对多关系以及松耦合。大脑里有了这些概念，还不能帮你在模式派对上露一手吗？

恭喜!

你的团队刚刚赢下了建造Weather-O-Rama公司下一代
基于Internet气象观测站的合同。



Weather-O-Rama, Inc.
100 Main Street
Tornado Alley, OK 45021

工作说明

恭喜贵公司被选中来建造我们的下一代基于Internet的气象观测站!

该气象站基于我们正在申请专利的WeatherData对象, 该对象追踪当前的天气状况(温度、湿度和气压)。我们希望贵公司创建一个应用, 该应用一开始提供三个显示元素: 当前状况、气象统计以及简单的预报。当WeatherData对象获得最新的测量数据时, 三个显示元素都必须实时更新。

而且, 这是一个可扩展的气象站。Weather-O-Rama希望允许其他开发人员编写自己的气象显示方式, 并插入到恰当位置。因此, 将来要便于添加新的显示方式, 这一点很重要。

Weather-O-Rama气象站认为我们有很棒的商业模式: 一旦客户上钩, 我们就对他们使用的每个显示收费。最棒的一点来了: 我们打算付给你们期权。

我们期待看到你的设计和应用的alpha版本。

真挚的,

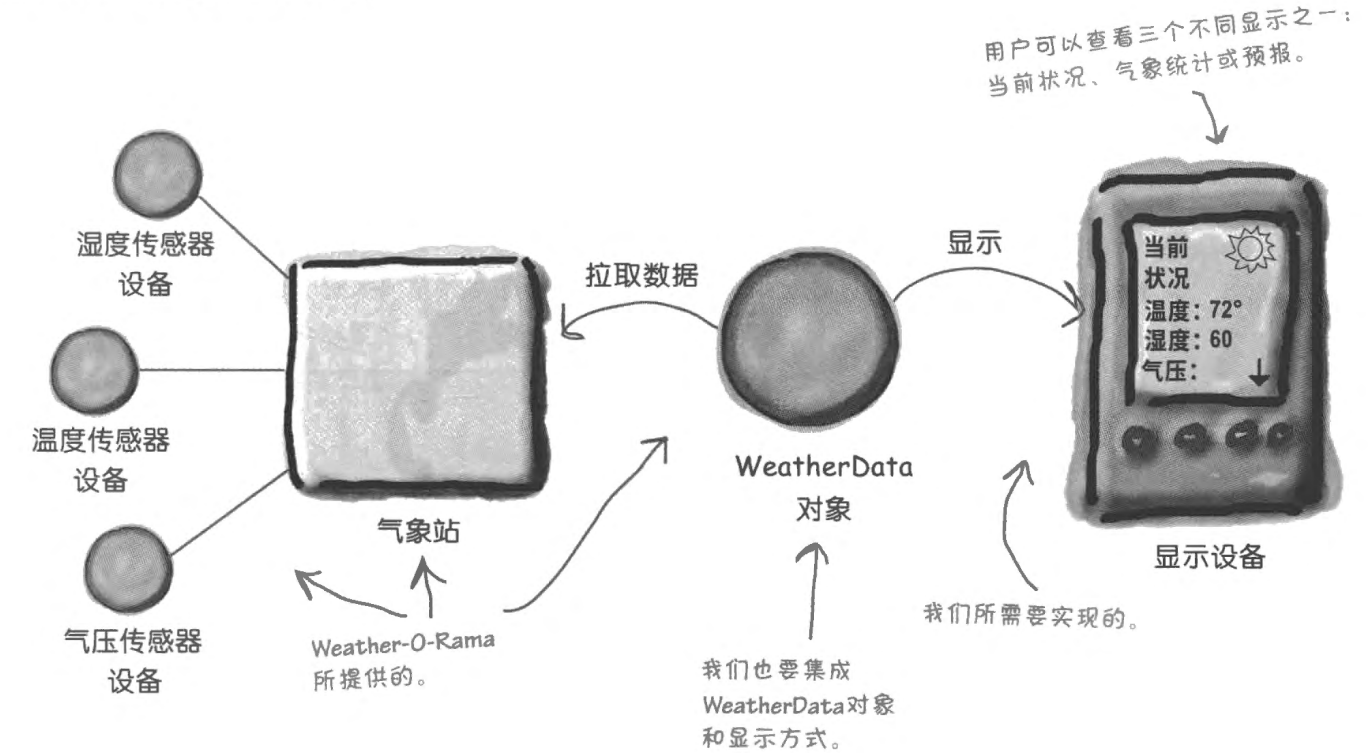
A handwritten signature in cursive script that reads "Johnny Hurricane".

Johnny Hurricane, CEO

另, 附件是WeatherData的源文件!

气象监测应用的概况

我们看看需要交付的气象监测应用—包括 Weather-O-Rama给我们的要求，以及我们打算建造或扩展的。系统有三个组件：气象站（获取实际气象数据的物理设备）、WeatherData对象（追踪来自气象站的数据并更新显示），以及向用户展示当前气象状况的显示方式。



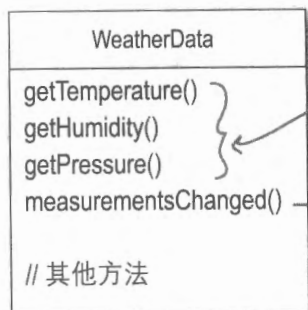
WeatherData对象由Weather-O-Rama编写，它知道如何与物理的气象站打交道，以获得更新的气象数据。我们需要适配WeatherData对象，以便它知道如何更新显示。但愿Weather-O-Rama在源代码给了我们提示如何做。记住，我们负责实现三个不同的显示元素：当前状况（显示温度、湿度和气压）、气象统计和简单的预报。

因此，如果我们选择接受这个任务，我们的工作就是创建一个应用，它使用WeatherData对象来更新三个显示：当前状况、气象统计和预报。

剖析WeatherData类

我们来检查一下CEO Johnny Hurricane发过来的源代码附件。从WeatherData类开始：

这是我们的WeatherData类。



这三个方法返回最近的气候测量，分别为温度、湿度和气压。
我们不在乎它现在如何获得这个数据，我们只知道WeatherData对象从气象站获得更新信息。
注意，无论何时WeatherData更新了值，measurementsChanged()方法被调用。

我们来看看measurementsChanged()方法，再一次，任何时候WeatherData获得新的温度、湿度和气压值，该方法会被调用。

```
/*
 * 任何时候气象测量被更新，
 * 这个方法被调用
 *
 */
public void measurementsChanged() {
    // 你的代码加在这里
}
```

WeatherData.java

即将要实现的显示。



显示设备

看起来，Weather-O-Rama在注释里留了一个提示，把我们的代码添加在那里。因此，这可能是我们需要更新显示的地方（一旦我们实现了它）。

因此，我们的工作，是改变measurementsChanged()方法，以便它更新当前状况的三个显示，还有气象统计和预报。

我们的目标

我们知道需要实现一个显示，每一次WeatherData有新值，或者换句话说，每一次measurementsChanged()方法被调用时，WeatherData就更新这个显示。但怎么做呢？我们来全面考虑一下我们在尝试的做法：

- 我们知道WeatherData类有三个测量值的getter方法：温度、湿度和气压。
- 我们知道，任何时候如果有了新的气象测量数据，measurementsChanged()方法会被调用（再说一次，我们不知道，或者说，不介意这个方法如何被调用；我们只知道它被调用就够了）。
- 我们需要实现三个使用气象数据的显示元素：一个当前状况显示，一个统计显示和一个预报显示。一旦WeatherData有新的测量，这些显示必须被更新。
- 为了更新显示，我们将添加代码到measurementsChanged()方法。



延展目标

我们也要考虑未来，还记得软件开发的不变因素吗？变化。我们期望，如果气象站成功，将来会有多于三个显示，因此为什么不为额外添加的显示创建一个市场？因此，这样建造怎么样：

- 可扩展性，其他开发人员可能要创建新的定制显示。为什么不允许用户添加（或移除）尽可能多的显示元素？当前，我们知道初始的三个显示类型（当前状况、统计和预报），但我们期望将来有一个生机勃勃的新显示市场。



气象站的第一个实现，有误导

这是第一个可能的实现——正如我们讨论过的，我们打算把我们的代码添加到WeatherData类的measurementsChanged()方法：

```
public class WeatherData {
```

```
// 实例变量声明
```

```
public void measurementsChanged() {
```

```
    float temp = getTemperature();  
    float humidity = getHumidity();  
    float pressure = getPressure();
```

```
    currentConditionsDisplay.update(temp, humidity, pressure);  
    statisticsDisplay.update(temp, humidity, pressure);  
    forecastDisplay.update(temp, humidity, pressure);
```

```
}
```

```
// 其他WeatherData方法在这里
```

```
}
```

此处是measurementsChanged()方法。

此处是我们添加的代码……

首先，我们通过调用WeatherData的getter方法获取最近的测量，然后把每个值赋给合适名称的变量。

接下来，我们打算更新每个显示……

……通过调用其update方法并传送给它最新的测量。



削尖你的铅笔

针对我们的一个实现，以下选项正确的是？（多选）

- ☐ A. 我们针对具体实现编程，而不是接口。
- ☐ B. 对于每个新的显示，我们都得修改这份代码。
- ☐ C. 我们没有办法在运行时添加(或移除)显示元素。
- ☐ D. 显示元素没有实现一个共同的接口。
- ☐ E. 我们还没有封装改变的部分。
- ☐ F. 我们侵犯了WeatherData类的封装。

我们的实现到底有什么问题？

回想第1章的所有概念和原则，哪一条我们违反了，哪一条没有违反？特别是，思考改变这段代码的后果。看着代码，我们来推敲一下：

```
public void measurementsChanged() {
```

我们再看一次……

```
    float temp = getTemperature();  
    float humidity = getHumidity();  
    float pressure = getPressure();
```

看起来像一个变化的区域。我们需要封装这里。

```
    currentConditionsDisplay.update(temp, humidity, pressure);  
    statisticsDisplay.update(temp, humidity, pressure);  
    forecastDisplay.update(temp, humidity, pressure);  
}
```

由于针对具体实现编程，我们没有办法在不修改代码的情况下添加或移除其他显示元素。

至少，我们看起来在使用一个通用的接口来和显示元素打交道……它们都有一个 `update()` 方法，参数都是 `temp`、`humidity` 和 `pressure` 的值。

如果我们要在运行时添加或移除显示，会怎么样？这看起来是硬编码。

好吧，我知道我是新来的，但考虑到我们目前在观察者模式这一章里面，也许我们应该开始使用它了？

好主意。我们看看观察者，然后回来找出把它用到气象监测应用的办法。

认识观察者模式

你知道报纸或杂志是怎么订阅的：

- ❶ 报社开始运营，出版报纸。
- ❷ 你向特定报社订阅，每次他们有新报纸，就交付给你。只要保持订阅，你就会一直得到新报纸。
- ❸ 当你不想再看报纸时，就取消订阅，报纸就停止交付。
- ❹ 只要报社还在运营，就会一直有人、酒店、航班和其他企业订阅和取消订阅报纸。

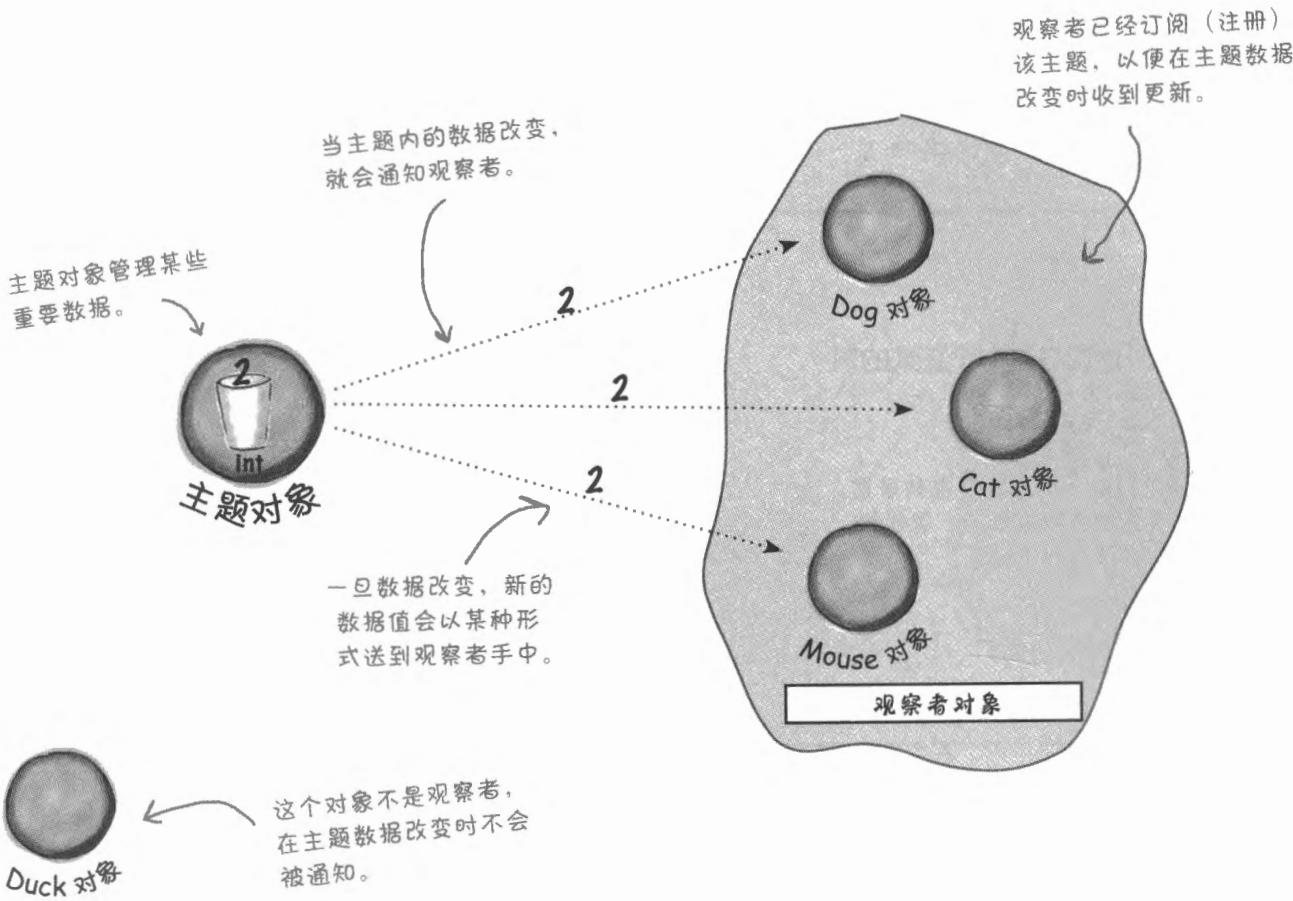
我们可不想错过对象村的各种事情，当然要订阅。



出版者 + 订阅者 = 观察者模式

如果你了解报纸的订阅，你就会在很大程度上理解观察者模式，只是我们把出版者叫作主题（SUBJECT），把订阅者叫作观察者（OBSERVER）。

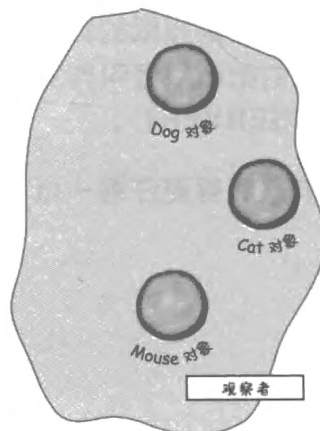
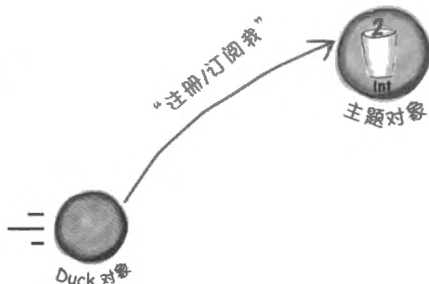
我们来看得更仔细一点：



观察者模式的一天

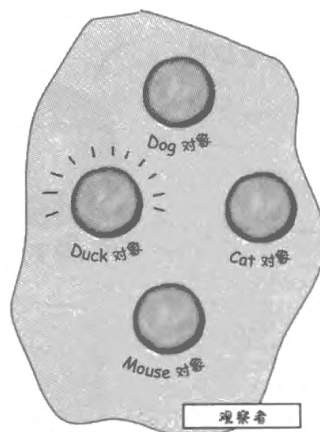
Duck对象过来告诉主题，想当一个观察者。

Duck真的想插一杠，这些Subject在其状态变化时发过来的int看起来相当有趣……



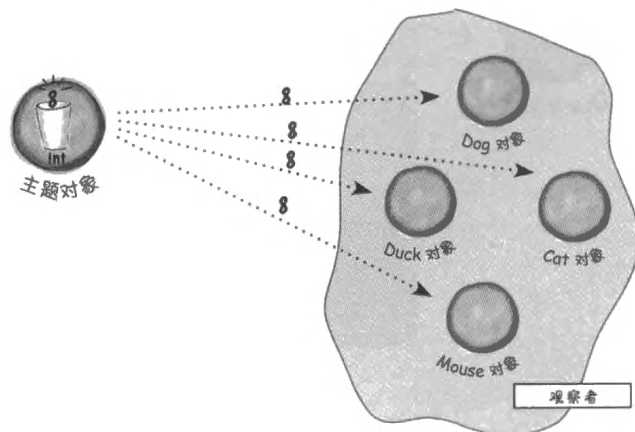
Duck对象现在是一个正式的观察者了。

Duck很激动……他现在在列表里，正在满怀期待地等着下一次通知，这样他会得到一个int。



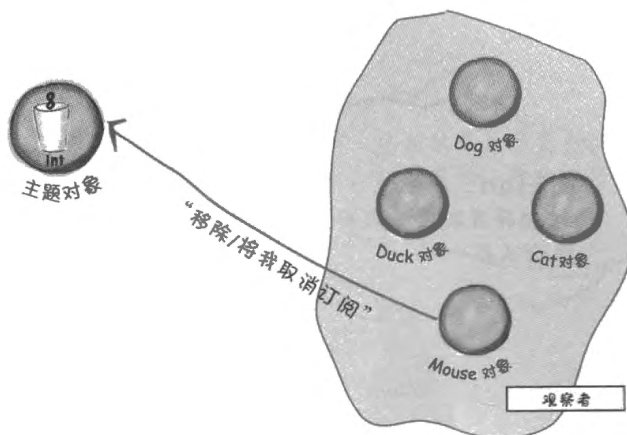
主题有新的数据值了！

现在，Duck以及其他所有观察者都会收到通知：Subject改变了。



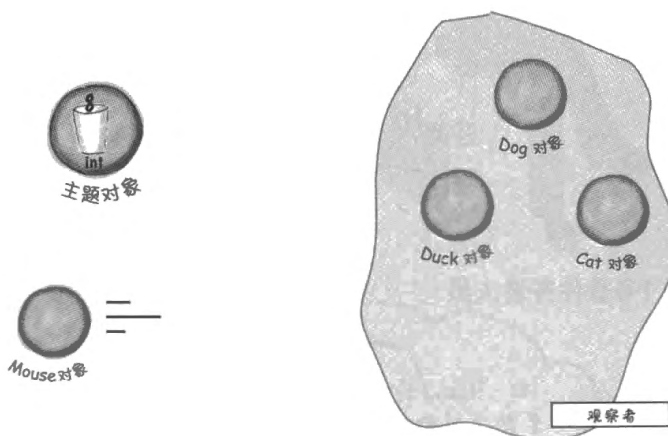
Mouse对象请求被移除，不再作为观察者。

Mouse对象已经得到了int一段时间了，厌倦了，因此他决定，是时候停止当观察者了。



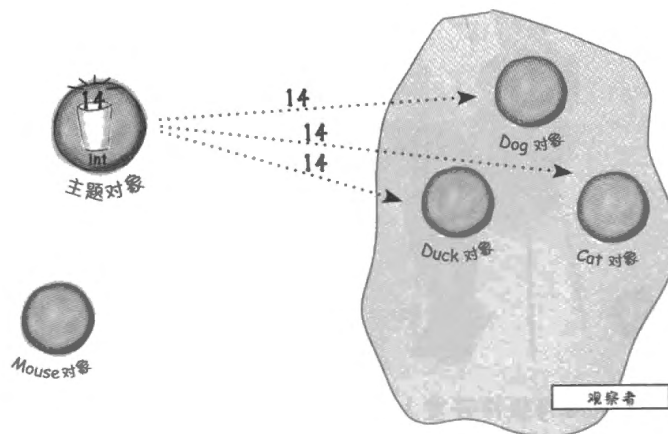
Mouse闪了!

Subject 认可了Mouse的请求并从观察者集合中移除了他。



Subject有了另一个新的int。

所有观察者得到了另一个通知，除了Mouse，他不再有份了。别告诉任何人，但Mouse偷偷地怀念这些int……也许某一天他会再次请求成为观察者。





五分钟短剧：用于观察的主题

在今天的讽刺短剧中，有两个软件开发人员，遇到一个真正的猎头……

我是Lori。我在找一个
Java开发职位。我有五年
经验以及……



1

1号软件开发人员

好的，宝贝。我把你和
其他人都加入我的**Java**开发
人员清单，不要打电话给我，
我会打电话给你！



2

猎头/主题

你好，我是Jill。我写过很
多企业系统。我对你那边的
Java开发工作感兴趣。



3

2号软件开发人员

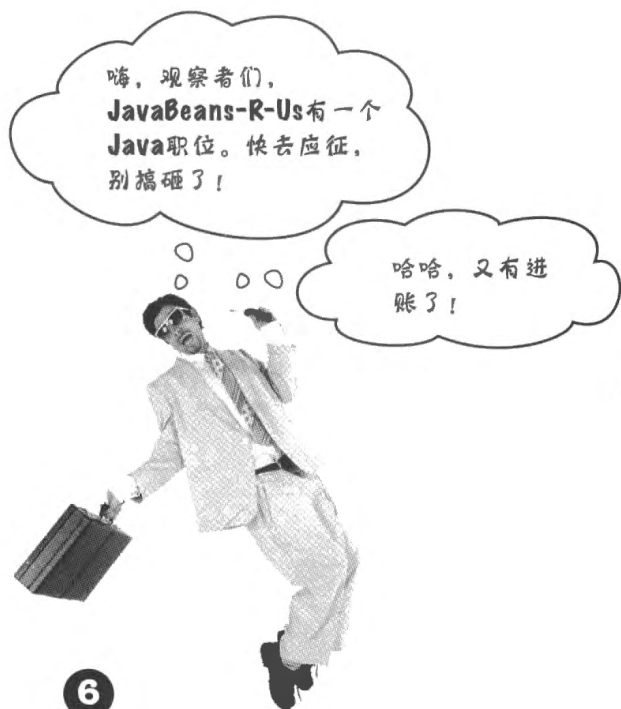
我会把你加入我的清单，
你会和其他人一样收到
我的通知。



4

主题

5 与此同时，Lori和Jill继续过自己的日子，如果Java工作来了，他们会被通知，毕竟，他们是观察者嘛！



主题

7

观察者



观察者

Jill自己找到工作了！



观察者



主题

两周以后……



Jill热爱她现在的生活，她不再是观察者了。她还获得了一笔签约费，因为公司不必付钱给猎头。

但是，我们亲爱的Lori怎么样了？我们听说她把那个猎头打趴了。她不只是一个观察者，还有了自己的求职者清单，她正在通知自己的观察者。Lori既是一个主题，也是一个观察者，集两种角色于一身。



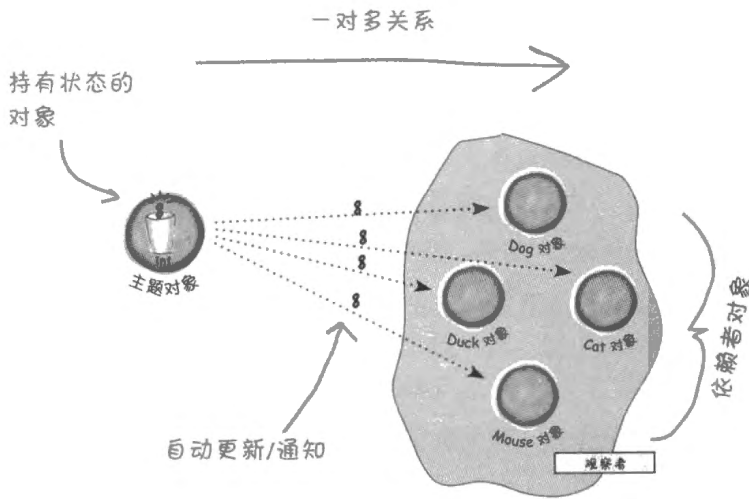
定义观察者模式

报纸订阅以及出版者和订阅者，是形象化观察者模式的一种好的方式。

在现实世界中，但是，你通常会看到观察者模式的定义像这样：

观察者模式定义对象之间的一对多依赖，这样一来，当一个对象改变状态时，它的所有依赖者都会收到通知并自动更新。

我们来把这个定义和我们之前对该模式的想法关联：



主题和观察者定义了一对多关系。当主题有变化时，会通知众多观察者。观察者是主题的依赖者，当主题的状态变化，观察者被通知。

稍后你会发现，观察者模式有几种不同的实现方式，但大多数围绕着包括主题和观察者接口的类设计。

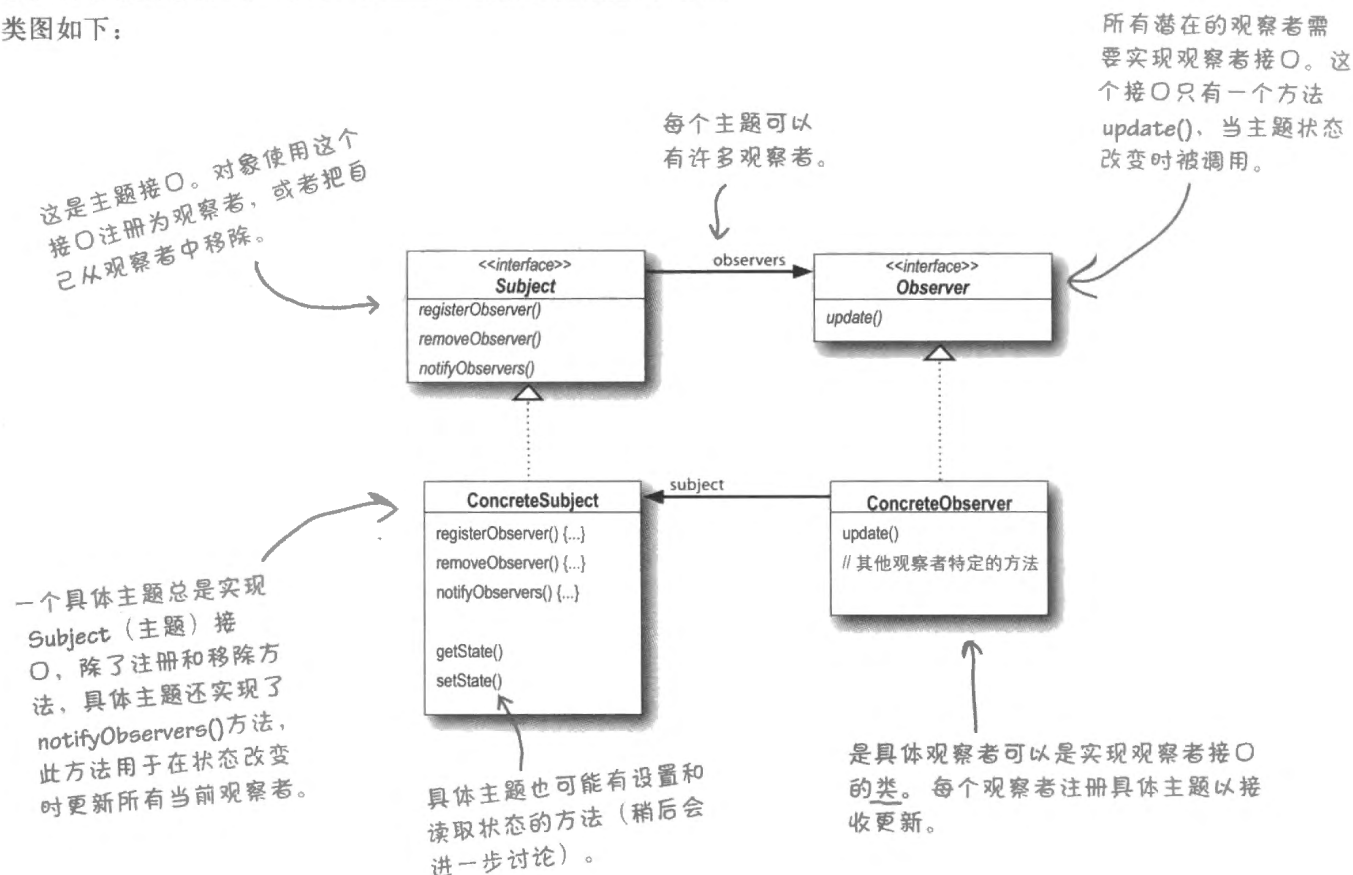
观察者模式定义一系列对象之间的一对多关系。

当一个对象改变状态，它的依赖者都会被通知。

观察者模式：类图

我们看看观察者模式的结构，把主题和观察者类完整列出来。

类图如下：



没有蠢问题

问：为什么必须用一对多关系？

答：对于观察者模式，主题是包含状态并控制状态的对象。因此，有一个带有状态的主题。另一方面，观察者使用这些状态，尽管这些状态不属于它们。有许多观察者，依赖主题告诉它们，状态何时改变了。所以有一个主题和多个观察者之间的关系。

问：依赖是怎样的？

答：因为主题是数据的唯一拥有者，观察者是主题的依赖者，在数据变化时更新，这样比起允许许多对象控制同一份数据来，可以得到更干净的 OO 设计。

问：我也听说过出版—订阅模式。它是观察者模式的另一个名称吗？

答：不是，虽然它们相关。出版—订阅模式是一个更复杂的模式，它允许订阅者对不同类型的消息感兴趣并进一步分离出版者和订阅者。这个模式经常用于中间件系统。



大师和徒弟……

大师：我们说过松耦合了吗？

徒弟：大师，我不记得讨论过。

大师：一只编织紧密的篮子是硬的还是有弹性的？

徒弟：硬的，大师。

大师：硬的还是有弹性的篮子，哪一个更不容易裂开或碎掉？

徒弟：有弹性的篮子不那么容易碎。

大师：那在我们的软件中，如果我们的对象绑在一起的紧密程度小一些，我们的设计是不是不那么容易碎？

徒弟：大师，我看到真相了。但对象绑得不那么紧是什么意思？

大师：我们喜欢把它叫作松耦合。

徒弟：啊哈！

大师：当有太多依赖到另一个对象时，我们说一个对象紧耦合到另一个对象。

徒弟：因此，一个松耦合对象不能依赖于另一个对象？

大师：想想自然界；所有生命都彼此依赖。同样，所有对象都依赖于其他对象。但一个松耦合对象不知道或在意另一个对象的太多细节。

徒弟：但是大师，这听起来并不是好事。不懂得肯定比懂得要更糟。

大师：你的研究做得不错，不过还有很多东西要学。通过不懂得其他对象太多的事情，我们可以创建更善于处理变化的设计，更有弹性的设计，就像编织不那么紧密的篮子。

徒弟：当然，我确定您是对的。您能给一个例子吗？

大师：改日再说。

松耦合的威力

当两个对象之间松耦合时，它们可以交互，但是通常对彼此所知甚少。正如我们将看到的，松耦合设计经常给我们带来很多弹性（不止一点点）。而且，正如结果所示，观察者模式是松耦合的一个很棒的例子。我们来看看，该模式如何达到松耦合：

首先，主题知道观察者的唯一一件事情是，它实现了某个接口（观察者接口）。它不需要知道观察者的具体类，它做什么或者关于它的其他事情。

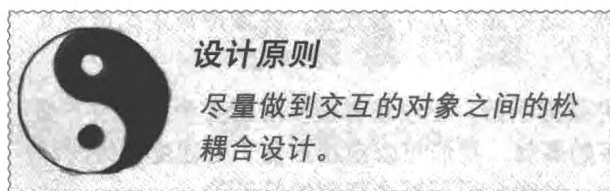
我们可以在任何时候添加新的观察者。因为主题唯一依赖的东西是实现观察者接口的对象列表，我们可以在想要添加的时候，添加新的观察者。事实上，我们可以在运行时替换任意观察者为另一个观察者，主题不会受到影响。同样，我们可以在任何时候移除观察者。

要添加新的类型的观察者，我们绝对不需要修改主题。假如我们有个新的具体类需要当观察者，并不需要对主题的代码做任何修改来兼容新的类型；我们只需要在新类中实现观察者接口并注册为一个观察者。主题不关心这个，它会发通知给所有实现了观察者接口的对象。



我们可以彼此独立地复用主题或观察者。如果我们在其他地方需要使用主题或观察者，可以轻易地复用，因为二者并非紧耦合。

改变主题或观察者其中一方，不会影响另一方。因为二者是松耦合的，所以我们可以自由地改变任一方，只要对象依然满足约定，实现主题或观察者接口。



看！我们有了一个新的设计原则！

松耦合设计允许我们建造能够应对变化的，有弹性的OO系统，因为对象之间的互相依赖降到最低。



削尖你的铅笔

在继续后面的内容之前，请试着画出实现气象站所需要的类，包括WeatherData类及其显示元素。确定你的图展示了各个部分如何结合起来，并允许其他开发人员实现他自己的显示元素。

如果你需要一点帮助，请阅读下一页，你的队友正在讨论如何设计气象站。

办公室隔间对话

回到气象站项目。你的队友们已经开始全面思考这个问题……



因此，我们打算怎样建造这个东西呢？

Mary: 嗯，我们在使用观察者模式。

Sue: 没错……但我们怎么应用？

Mary: 嗯。我们再来看看定义：

观察者模式定义对象之间的一对多依赖，这样一来，当一个对象改变状态时，它的所有依赖者都会被通知并自动更新。

Mary: 这个定义想起来确实有些道理。我们的WeatherData类是“一”，我们的“多”是使用气象测量的各种显示元素。

Sue: 没错。WeatherData对象当然有状态，就是温度、湿度、气压，而这些肯定会变化。

Mary: 对的，当这些测量值变化时，我们不得不通知所有显示元素，以便它们可以根据测量值做相应处理。

Sue: 酷，现在我想我明白观察者模式如何应用到我们的气象站问题上了。

Mary: 还有一些东西要考虑一下，我还不能确定我理解了。

Sue: 比如？

Mary: 其中一个，我们如何把气象观测值给显示元素？

Sue: 唔，回头看看观察者模式的图，如果我们把WeatherData对象当作主题，把显示元素当作观察者，那么，为了获得所需信息，显示会向WeatherData对象注册自己。

Mary: 是的……一旦气象站知道了一个显示元素，它只需要调用一个方法来告诉测量数据。

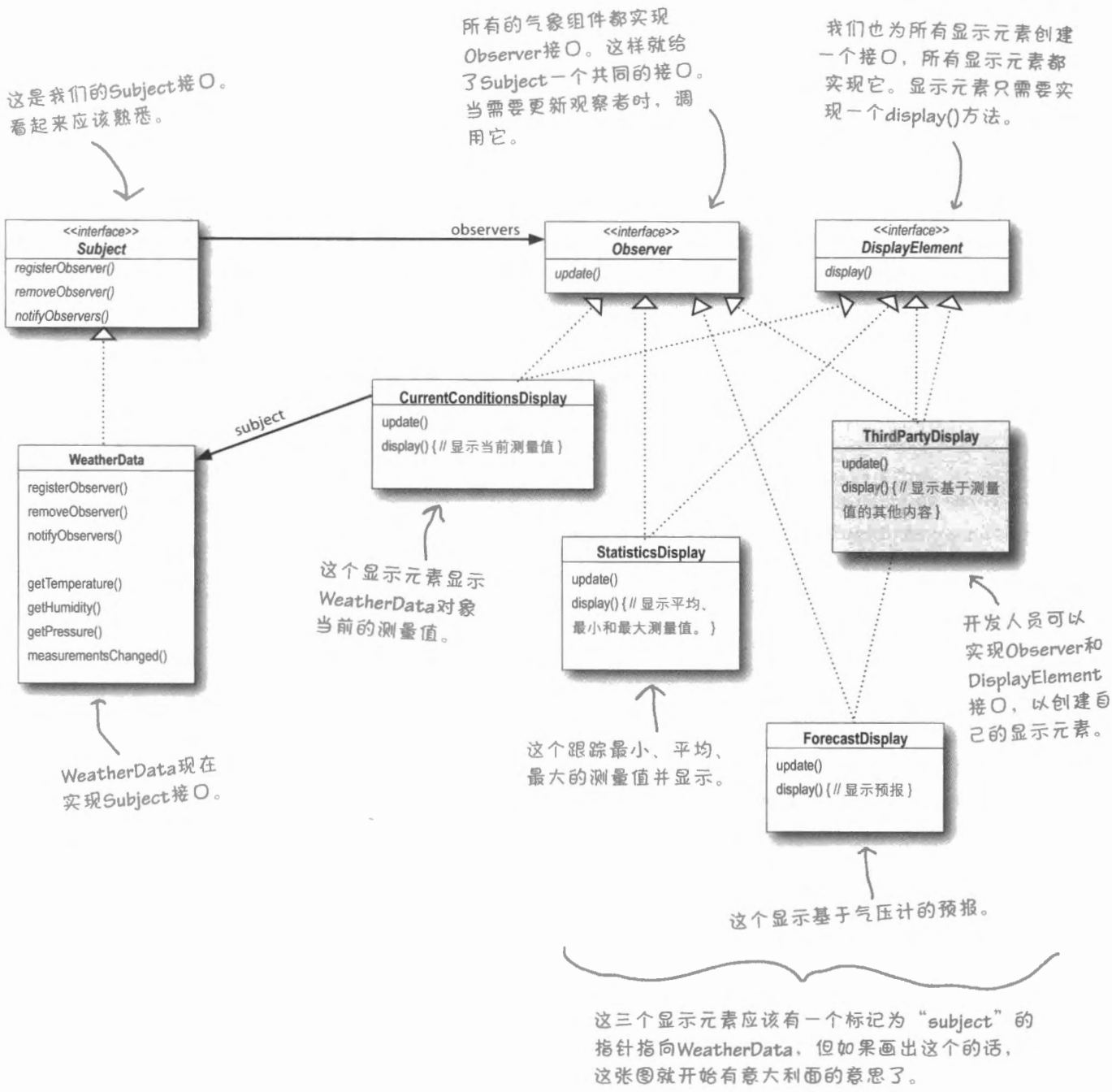
Sue: 我们要记住，每个显示元素可以不同……因此我想这就是要有一个共同接口的由来。即使每个组件有不同的类型，它们应该都实现相同的接口，这样WeatherData对象知道如何把测量值发给它们。

Mary: 我知道你的意思了。所以每个显示会有一个大概名为update()的方法，以供WeatherData对象调用。

Sue: update()在所有元素都实现的共同接口中定义……

设计气象站

这个图和你的相比怎么样？



实现气象站

好，我们从Mary和Sue那里（前面几页）得到了一些很棒的想法，我们有了一张详细描述类整体结构的图。因此，我们现在来实现气象站。我们从接口开始：

```
public interface Subject {
    public void registerObserver(Observer o);
    public void removeObserver(Observer o);
    public void notifyObservers();
}
```

这两个方法都用一个Observer作为参数，即要注册或被移除的Observer。

当Subject的状态改变时，这个方法会被调用，以通知所有的观察者。

```
public interface Observer {
    public void update(float temp, float humidity, float pressure);
}
```

这些是当气象测量数据变化时观察者从Subject获得的状态值。

所有的观察者都实现Observer接口，所以它们都不得不实现update()方法。在这里，我们遵照Mary和Sue的看法，把测量值传给观察者。

```
public interface DisplayElement {
    public void display();
}
```

DisplayElement接口只包含一个方法display()，当显示元素需要显示时，调用此方法。



Mary和Sue认为：把测量值直接转给观察者，是更新状态的最直接方法。你认为这样做明智吗？暗示：该应用是否有某个区域将来会变化？如果确实会变化，这些变化是否被很好地封装？或者需要修改许多代码才能办到？

关于传送更新后的状态给观察者，你能否想到其他的方法来解决问题吗？

别担心，在完成第一次实现后，我们会回来探讨这个设计决策。

在WeatherData中实现主题接口

还记得我们在本章一开始第一次试图实现WeatherData类吗？你可能需要刷新一下记忆。现在，是时候回去，带着观察者模式的思想干活了：

记住：我们没有在代码列表中提供import和package语句。完整的源代码可以从<https://wickedlysmart.com/head-first-design-patterns>下载

我们在这里实现Subject接口。

```

public class WeatherData implements Subject {
    private List<Observer> observers;
    private float temperature;
    private float humidity;
    private float pressure;

    public WeatherData() {
        observers = new ArrayList<Observer>();
    }

    public void registerObserver(Observer o) {
        observers.add(o);
    }

    public void removeObserver(Observer o) {
        observers.remove(o);
    }

    public void notifyObservers() {
        for (Observer observer : observers) {
            observer.update(temperature, humidity, pressure);
        }
    }

    public void measurementsChanged() {
        notifyObservers();
    }

    public void setMeasurements(float temperature, float humidity, float pressure) {
        this.temperature = temperature;
        this.humidity = humidity;
        this.pressure = pressure;
        measurementsChanged();
    }

    // 其他WeatherData方法在这里
}

```

WeatherData现在实现Subject接口。

我们添加一个ArrayList来持有Observer, 并在构造器中创建它。

当一个观察者注册时，我们只是把它添加到列表末端。

同样，当一个观察者想要取消注册时，我们只要把它从列表拿走。

有趣的部分在这里，在这个地方，我们把状态告诉所有观察者。因为它们都是Observer，我们知道它们都实现了update()，所以我们知道如何通知它们。

当从气象站得到更新的测量值时，我们通知Observer。

OK，虽然我们想要每本书附带一个小型气象站，但是出版商不肯。因此，就无法从设备中读取实际的气象数据，我们也只好用这个方法测试显示元素。或者，为了乐趣，你也可以写代码从网站上抓取测量值。

现在，我们来建造那些显示元素

现在我们已经有了WeatherData类，是时候建造显示元素了。Weather-O-Rama 订购了三个：当前状况显示、统计显示和预报显示。我们看看当前状况显示，一旦你熟悉了显示元素，就可以在代码目录中找到统计和预报显示的代码。你会看到，这些代码很类似。

这个显示实现了Observer接口，所以它可以从WeatherData对象中获取变化。

它也实现了DisplayElement，因为我们的API打算要求所有显示元素实现这个接口。

```
public class CurrentConditionsDisplay implements Observer, DisplayElement {
    private float temperature;
    private float humidity;
    private WeatherData weatherData;
```

构造器被传入weatherData对象（Subject），我们用它来把显示注册为观察者。

```
    public CurrentConditionsDisplay(WeatherData weatherData) {
        this.weatherData = weatherData;
        weatherData.registerObserver(this);
    }
```

```
    public void update(float temperature, float humidity, float pressure) {
        this.temperature = temperature;
        this.humidity = humidity;
        display();
    }
```

当update()被调用，我们保存温度和湿度并调用display()。

```
    public void display() {
        System.out.println("Current conditions: " + temperature
            + "F degrees and " + humidity + "% humidity");
    }
}
```

display()方法只是把最近的温度和湿度显示出来。

没有蠢问题

问： update()是最适合调用display()的地方吗？

答： 在这个简单的例子中，当值变化的时候调用display()，是很合理的。但是，你是对的，显示数据有更好的方法。当我们谈到模型-视图-控制器（Model-View-Controller）模式时，会了解这一点。

问： 为什么保存对WeatherData主题的引用？在构造器中使用之后，以后似乎用不着了呀？

答： 的确如此，但是将来可能需要取消作为观察者的注册，如果有对主题的引用会比较方便。

启动气象站



❶ 首先，我们来创建一个测试程序。

气象站扬帆待发。我们只需要一些代码把每样东西粘合起来。我们添加更多一些显示，并做进一步泛化。下面是我们的第一次尝试：

```
public class WeatherStation {

    public static void main(String[] args) {
        WeatherData weatherData = new WeatherData();

        CurrentConditionsDisplay currentDisplay =
            new CurrentConditionsDisplay(weatherData);
        StatisticsDisplay statisticsDisplay = new StatisticsDisplay(weatherData);
        ForecastDisplay forecastDisplay = new ForecastDisplay(weatherData);

        weatherData.setMeasurements(80, 65, 30.4f);
        weatherData.setMeasurements(82, 70, 29.2f);
        weatherData.setMeasurements(78, 90, 29.2f);
    }
}
```

如果你不想下载代码，可以将这两行注释掉并运行。

首先，创建 WeatherData 对象。

创建三个显示并传给它们 WeatherData 对象。

模拟新的气象测量。

❷ 运行代码，让观察者模式表演魔术。

```
File Edit Window Help StormyWeather
%java WeatherStation
Current conditions: 80.0F degrees and 65.0% humidity
Avg/Max/Min temperature = 80.0/80.0/80.0
Forecast: Improving weather on the way!
Current conditions: 82.0F degrees and 70.0% humidity
Avg/Max/Min temperature = 81.0/82.0/80.0
Forecast: Watch out for cooler, rainy weather
Current conditions: 78.0F degrees and 90.0% humidity
Avg/Max/Min temperature = 80.0/82.0/78.0
Forecast: More of the same
%
```



削尖你的铅笔

Johnny Hurricane, Weather-O-Rama的CEO, 刚打来电话说他们还需要热指数显示元素, 否则无法上线。以下是细节。

热指数是一个结合温度和湿度的指数, 用来表征人的温度感受 (实际上感觉到有多热)。要计算热指数, 可以用温度T和相对湿度RH来套用以下公式:

heatindex =

$$\begin{aligned}
 &16.923 + 1.85212 * 10^{-1} * T + 5.37941 * RH - 1.00254 * 10^{-1} * \\
 &T * RH + 9.41695 * 10^{-3} * T^2 + 7.28898 * 10^{-3} * RH^2 + 3.45372 * \\
 &10^{-4} * T^2 * RH - 8.14971 * 10^{-4} * T * RH^2 + 1.02102 * 10^{-5} * T^2 * \\
 &RH^2 - 3.8646 * 10^{-5} * T^3 + 2.91583 * 10^{-5} * RH^3 + 1.42721 * 10^{-6} \\
 &* T^3 * RH + 1.97483 * 10^{-7} * T * RH^3 - 2.18429 * 10^{-8} * T^3 * RH^2 \\
 &+ 8.43296 * 10^{-10} * T^2 * RH^3 - 4.81975 * 10^{-11} * T^3 * RH^3
 \end{aligned}$$

开始练习打字吧!

开玩笑的。别担心, 你不必敲这个公式, 只要创建你自己的`HeatIndexDisplay.java` 文件并从`heatindex.txt`中把公式复制进去。 ←

你可以从wickedlysmart.com获得`heatindex.txt`。

这个公式是怎么工作的? 你可以参考《Head First Meteorology》, 或者问问国家气象局的同志 (或者尝试在Web上搜)。

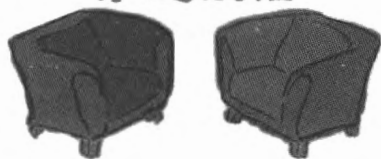
当你完成时, 输出结果应如下所示:

输出在这里
有变化。

```

File Edit Window Help OverDaRainbow
%java WeatherStation
Current conditions: 80.0F degrees and 65.0% humidity
Avg/Max/Min temperature = 80.0/80.0/80.0
Forecast: Improving weather on the way!
Heat index is 82.95535
Current conditions: 82.0F degrees and 70.0% humidity
Avg/Max/Min temperature = 81.0/82.0/80.0
Forecast: Watch out for cooler, rainy weather
Heat index is 86.90124
Current conditions: 78.0F degrees and 90.0% humidity
Avg/Max/Min temperature = 80.0/82.0/78.0
Forecast: More of the same
Heat index is 83.64967
%
```

炉边谈话



今夜话题：主题和观察者争论把状态信息给观察者的正确方式。

主题：

我很高兴，我们终于有机会面对面聊天了。

好吧，我做了该做的事，不是吗？我总是告诉你们发生什么事了……虽然我不知道你们是谁，但这并不意味着我不在乎。另外，我知道关于你们的一件最重要的事情：你们实现了Observer接口。

是吗，比如？

抱歉！我不得不发送我的状态以及通知，这样你们这些懒惰的观察者才知道发生了什么！

嗯……这样可能可以。但是，我不得不公开更多东西，以便让你们所有观察者进来取得你们要的状态。这可能会有危险。我不能让你们进来到处乱逛，看到我所有的东西。

观察者：

真的？我以为你不太关心我们观察者。

是的，但这只是关于“我是谁”的一小部分。无论如何，我对你懂得更多……

好吧，你总是把你的状态传给我们观察者，所以我们可以看到你内部的进展。有时候，这很烦人……

OK，等一会。首先，我们不懒，在你所谓“很重要”通知的空挡中，我们还有别的事要做，主题先生；其次，为什么你不让我们找你要我们想要的状态，而只是把它推送给每个人？

主题：

是的，我可以让你们拉走我的状态，但是这样对你们不是更不方便吗？如果每次你想要某些东西时都不得不来找我，你可能不得不做多次方法调用来获得所有想要的状态。这就是为什么我更喜欢推……你们可以在一个通知中获得需要的每样东西。

哇哦，就像我喜欢说的，不要打电话给我，我们会打电话给你！但我会考虑一下的。

你绝对不知道的，地狱都能结冰。

确实。

观察者：

为什么你不只是写一些公开的getter方法，以便让我们拉取我们需要的状态？

不要那么固执！我们观察者有如此多不同的种类，你没有办法预测我们所需的每样东西。就让我们去你那里去获取我们所需的状态吧。这样，如果我们中的某些人只需要状态的一小点，就不会强迫收到所有数据。后面的修改也更容易了。比如说，要是你扩展自身，添加更多的状态，如果你用拉取方式，你不必到处查看并修改每个观察者上的更新调用；你只需要改变自身为允许更多的getter方法访问我们额外添加的状态。

我对此不抱什么期望。

我明白了，自作聪明的家伙……

在荒野中寻找观察者模式

观察者模式是最常用的模式之一。观察许多库和框架，你会发现其中应用该模式的大量例子。例如，如果我们看一看Java开发工具包（JDK），会发现JavaBean和Swing库都使用观察者模式。该模式不局限于Java，还用在JavaScript的事件、Cocoa和Swift的Key-Value Observing协议等。懂得设计模式的好处之一，就是能识别并快速理解你喜爱的库的设计动机。我们花点时间快速看一下Swing库，看看观察者是怎样使用的。



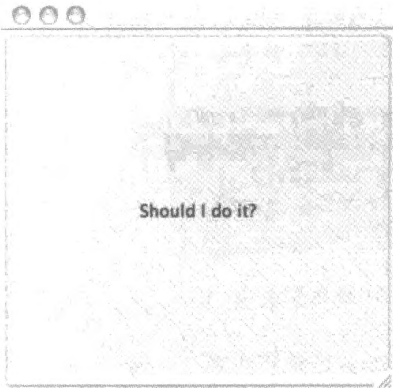
关于JavaBean中的观察者模式可以看一下PropertyChangeListener接口。

Swing库

你可能已经知道，Swing是Java的用户界面（GUI）工具箱。这个工具箱中的一个最基本的组件是JButton类。如果仔细看看JButton的超类AbstractButton，你会发现它有许多添加/移除监听器的方法。这些方法允许你添加和移除观察者[或者，如Swing里面称呼的，监听器（listener）]来监听Swing组件上发生的各种类型的事件。例如，ActionListener“监听”按钮上可能发生的任何类型的动作，比如，按下按钮。你会发现Swing API各处都有各种类型的监听器。

一个小的、改变生活的应用

OK，我们的应用相当简单。有一个按钮，上面有“Should I do it?”（我该做吗），当你点击按钮，监听器（观察者）用它们的方式回答问题。我们实现两个这样的监听器，叫作AngelListener和DevilListener。应用的行为如下：



这是我们的很炫的接口。



这是点击按钮后的输出。

魔鬼的回答
天使的回答

```
File Edit Window Help HeMadeMeDolt
%java SwingObserverExample
Come on, do it!
Don't do it, you might regret it!
%
```

编码生活改变应用

这个生活改变应用所需代码很少。我们需要做的只是创建一个JButton对象，把它添加到JFrame，并设置我们的监听器。我们打算为监听器使用内部类，这是Swing编程中的常有技巧。如果你不熟悉内部类或Swing，可以复习你喜欢的Java参考资料里的Swing章节。

```
public class SwingObserverExample {
    JFrame frame;

    public static void main(String[] args) {
        SwingObserverExample example = new SwingObserverExample();
        example.go();
    }

    public void go() {
        frame = new JFrame();

        JButton button = new JButton("Should I do it?");
        button.addActionListener(new AngelListener());
        button.addActionListener(new DevilListener());

        // 在这里设置frame属性
    }
```

简单的Swing应用，只是创建一个JFrame，放进一个按钮。

做按钮的魔鬼和天使对象监听器（观察者）。

设置frame的代码在这里。

```
class AngelListener implements ActionListener {
    public void actionPerformed(ActionEvent event) {
        System.out.println("Don't do it, you might regret it!");
    }
}

class DevilListener implements ActionListener {
    public void actionPerformed(ActionEvent event) {
        System.out.println("Come on, do it!");
    }
}
```

这里是观察者的类定义，定义为内部类（但不是必须的。）

当主题（本例中为按钮）里的状态变化时，不调用update()，而是调用actionPerformed()方法。



Serious Coding

Java8中新增了Lambda表达式。如果你不熟悉，不必担心，你可以继续对你的Swing观察者使用内部类。



观察者模式进一步还能怎么用呢？通过使用lambda表达式而不是内部类，你可以省去创建ActionListener对象的步骤。有了lambda表达式，我们改为创建一个函数对象，该函数对象是一个观察者。当你传送这个函数对象给addActionListener()时，Java确保其签名和ActionListener接口唯一的方法actionPerformed()匹配。

后面，当按钮被点击时，按钮对象通知其观察者（包括由lambda表达式创建的函数对象），它被点击了，并调用每一个监听器的actionPerformed()方法。

我们来看看，如何使用lambda表达式作为观察者来简化我们之前的代码：

使用lambda表达式修改后的代码如下：

```
public class SwingObserverExample {
    JFrame frame;
    public static void main(String[] args) {
        SwingObserverExample example = new SwingObserverExample();
        example.go();
    }
    public void go() {
        frame = new JFrame();

        JButton button = new JButton("Should I do it?");
        button.addActionListener(event ->
            System.out.println("Don't do it, you might regret it!"));
        button.addActionListener(event ->
            System.out.println("Come on, do it!"));

        // 在这里设置frame的属性
    }
}
```

我们用实现和之前一样功能的lambda表达式替换AngelListener和DevilListener对象。

当你点击按钮时，由lambda表达式创建的函数对象得到通知，它们所实现的方法运行。

我们把两个ActionListener类（DevilListener和AngelListener）移除干净了。

使用lambda表达式使得这段代码简洁很多。

关于lambda表达式的更多内容，可参见Java的文档。

没有蠢问题

问：我想，Java有Observer和Observable类？

答：被你抓到了。Java过去常提供一个Observable类（Subject）和一个Observer接口，你可以用它们来帮忙在你的代码中整合观察者模式。Observable类提供方法来添加、删除和通知观察者，这样，你就不必编写这方面的代码。Observer接口提供一个和我们所做的相似的接口，带有一个update()方法。在Java 9中，不赞成使用这些类。人们发现在自己的代码中支持基本的观察者模式更容易，或者能让某些东西更稳定，因此Observer/Observable类正在被逐步淘汰。

问：Java提供为观察者其他内建的支持来替换那些类了吗？

答：JavaBean通过PropertyChangeEvent提供内建的支持。当一个Bean改变特定种类的属性时，该事件产生，并发送通知给PropertyChangeListener。在处理异步流的Flow API中，也有相关的出版者/订阅者组件。

问：我应该期望从主题到其观察者的通知以特定次序到达吗？

答：根据Java的观察者实现，JDK 开发人员特地建议你不要依赖于任何特定的通知次序。

我在想之前我们进行的推/拉讨论。如果我们允许显示从WeatherData拉取所需数据，代码是不是更通用一些？这可能使得将来添加新的显示更容易。

好主意。

在我们当前的气象站设计中，我们推送所有三种数据给显示的update()方法，即使该显示不需要这些值的全部。这没问题，但如果Weather-O-Rama后面添加另一个数据值，例如风速，怎么办？我们不得不修改所有显示中的所有update()方法，即使其中大多数不需要或不想要风速数据。

现在，我们是拉还是推数据给Observer，是一个实现细节，但在许多情况下，让Observer检索它们需要的数据，而不是通过update()方法传递越来越多的数据给它们，是有道理的。毕竟，随着时间的推移，会有某个地方发生变化并笨拙生长。而且我们知道 CEO Johnny Hurricane打算要扩展气象站，并售卖更多的显示，因此，我们来到了另一个设计的关口，看看我们是否可以使其在将来更易于扩展。

更新气象站代码，以允许Observer拉取所需要的数据，是一个相当简单明了的练习。我们需要做的确保Subject有数据的getter方法，然后把Observer修改为：使用它们来拉取适合其需要的数据。我们来做一下。





与此同时，回到Weather-O-Rama

有另外一种处理Subject中数据的方式：我们可以让Observer按需要从Subject拉取。当Subject的数据变化，我们马上通过调用update()传送数据，推送新的温度、湿度和气压值给Observer。

我们来设置一下，以便当一个Observer被通知变化时，它调用Subject上的getter方法来拉取所需的值。

为了切换到拉取方式，我们需要对已有代码做一些小小的修改。

Subject的发送通知……

- 1 我们修改 WeatherData中的notifyObservers() 方法，不带参数地调用Observer中的update()方法：

```
public void notifyObservers() {
    for (Observer observer : observers) {
        observer.update();
    }
}
```

Observer的接收通知……

- 1 然后，我们修改Observer接口，改变update()方法的签名，这样它就没有参数了：

```
public interface Observer {
    public void update();
}
```

- 2 最后，我们修改每个具体Observer，改变其各自的update()方法的签名，并使用 WeatherData的getter方法从Subject获取气象数据。 CurrentConditionsDisplay类的新代码如下：

```
public void update() {
    this.temperature = weatherData.getTemperature();
    this.humidity = weatherData.getHumidity();
    display();
}
```

在这里，我们使用 Subject的getter方法，该方法的代码由 Weather-O-Rama的 WeatherData提供。



代码贴

ForecastDisplay类的代码小贴纸在冰箱上被弄乱了。你能够重新排列代码碎片恢复原样吗？有些花括号掉到地上了，因为太小不好捡起来，所以如果你觉得需要花括号，可以自行加上！

```
public ForecastDisplay(WeatherData
weatherData) {
```

```
display();
```

```
weatherData.registerObserver(this);
```

```
public class ForecastDisplay implements
Observer, DisplayElement {
```

```
public void display() {
    // 这里显示代码
}
```

```
lastPressure = currentPressure;
currentPressure = weatherData.getPressure();
```

```
private float currentPressure = 29.92f;
private float lastPressure;
```

```
this.weatherData = weatherData;
```

```
public void update() {
```

```
}
```

```
private WeatherData weatherData;
```

试驾新代码



OK，你又有了一块要更新的显示，平均/最小/最大显示。赶紧去做吧！

为了确定没问题，我们来运行新代码……

这是我们已有的。



看！这是刚来的！



```
File Edit Window Help TryThisAtHome

%java WeatherStation
Current conditions: 80.0F degrees and 65.0% humidity
Avg/Max/Min temperature = 80.0/80.0/80.0
Forecast: Improving weather on the way!
Current conditions: 82.0F degrees and 70.0% humidity
Avg/Max/Min temperature = 81.0/82.0/80.0
Forecast: Watch out for cooler, rainy weather
Current conditions: 78.0F degrees and 90.0% humidity
Avg/Max/Min temperature = 80.0/82.0/78.0
Forecast: More of the same
%
```



Weather-O-Rama, Inc.
100 Main Street
Tornado Alley, OK 45021

哇哦！

你的设计非常棒。你不仅快速创建我们想要的所有三个显示，还创建了一个通用的设计，允许任何人创建新的显示，甚至允许用户在运行时添加或移除显示！

有才！

期待下一次合作，

Johnny Hurricane



你的设计工具箱中的工具

欢迎来到第2章末尾。你给你的OO工具箱添加了一些新东西……

OO基础

抽象

OO原则

封装变化。

组合优于继承。

针对接口编程，而不是针对实现。

尽力达到交互对象之间的松耦合设计。

这是你最新的原则。
记住，松耦合设计更有弹性，更能应对变化。

OO模式

Str
encap
inter
algor
that

观察者，定义对象之间的一对多依赖这样，当一个对象改变状态时，它的所有依赖者会被通知并自动更新。

一个以松耦合方式和一个对象集合沟通状态的模式等。我们还没有看到观察者模式的尽头，直到我们谈到MVC！

要点

- 观察者模式定义对象之间的一对多关系。
- 主题使用通用接口更新观察者。
- 任何具体类型的观察者都可以参与该模式，只要它们实现观察者接口。
- 观察者是松耦合的，除了知道它们实现观察者接口之外，主题对它们的其他事情不知情。
- 使用该模式时，你可以从主题推或拉数据（拉被认为更“正确”）。
- Swing大量使用观察者模式，许多GUI框架也是这样。
- 你也会在其他很多地方发现该模式，包括RxJava、JavaBeans和RMI，以及其他语言的框架，像Cocoa、Swift和JavaScript事件。
- 观察者模式和出版/订阅模式相关。出版/订阅模式用于更复杂得多主题和/或多消息类型的情形。
- 观察者模式是一个常用的模式，当我们学习模型-视图-控制器时，还会看到它。



设计原则挑战

对于每个设计原则，描述观察者模式如何利用该原则。

设计原则

识别应用中会变化的方面，并将其和不变的方面分离。

设计原则

针对接口编程，而不是针对实现。

设计原则

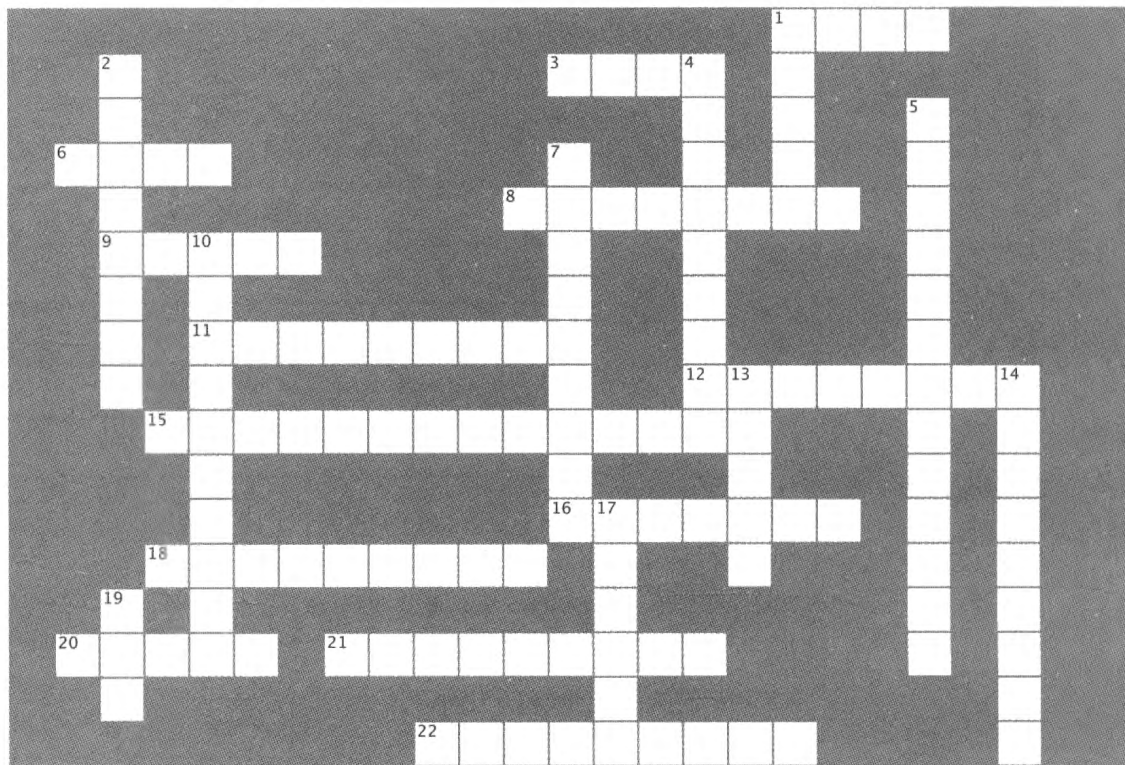
优先使用组合而不是继承。

这个问题比较难。提示：考虑观察者和主题如何一起工作。



设计模式填字游戏

是时候再让你的右脑做些事情了！所有词汇来自第1章和第2章。



横排

1. 主题喜欢和_____的观察者交谈。
3. 主题初始时要_____所有数据到观察者。
6. CEO几乎忘记_____索引显示。
8. `CurrentConditionsDisplay` 实现这个接口。
9. 带大量观察者的Java 框架。
11. 主题和_____类似。
12. 当某些事情发生时，观察者喜欢被_____。
15. 如何把你自己从观察者列表排除。
16. Lori是一个观察者和_____。
18. 主题是一个_____。
20. 你要保持你的耦合_____。
21. 针对_____编程，而不是实现。
22. 魔鬼和天使是对按钮的_____。

竖排

1. 他不再需要int，因此他移除自己。
2. 温度，湿度，以及_____。
4. Weather-O-Rama的CEO以这种风暴命名。
5. 他说你应该尽力试一试。
7. 主题 不必知道太多关于_____。
10. `WeatherData`类_____主题接口。
13. 不要依赖这个来通知。
14. 观察者是主题上的_____。
17. 实现这个方法以获得通知。
19. Jill得到她自己的_____。



削尖你的铅笔 答案

基于我们的第一个实现，以下哪些说法正确？
(多选)

- ☒ A. 我们在针对具体实现编程，而不是接口。
- ☒ B. 对于每个新的显示元素，我们需要修改代码。
- ☒ C. 我们无法在运行时添加显示元素。
- ☐ D. 显示元素没有实现通用接口。
- ☒ E. 我们还没有封装变化。
- ☐ F. 我们在侵犯WeatherData类的封装。



设计原则 挑战答案

设计原则

识别应用中会变化的方面，并将其和不变的方面分离。

在观察者模式中，改变的是主题的状态，以及观察者的数目和类型。有了这个模式，你可以改变依赖于主题状态的对象，却不必改变主题。这叫作提前规划！

设计原则

针对接口编程，而不是针对实现。

主题与观察者都使用接口。主题跟踪实现观察者接口的对象，而观察者通过主题接口注册并被通知。正如我们已经看到的，这样保持一切正常而且松耦合。

设计原则

优先使用组合而不是继承。

观察者模式使用“组合”来将任意数目的观察者组合进主题。这些关系不是由某种继承层次设置。不是的，它们在运行时通过组合设置！



代码贴答案

ForecastDisplay类的代码小贴纸在冰箱上被弄乱了。你能够重新排列代码碎片恢复原样吗？有些花括号掉到地上了，因为太小不好捡起来，所以如果你觉得需要花括号，可以自行加上！以下是答案。

```
public class ForecastDisplay implements  
Observer, DisplayElement {
```

```
    private float currentPressure = 29.92f;  
    private float lastPressure;
```

```
    private WeatherData weatherData;
```

```
    public ForecastDisplay(WeatherData  
weatherData) {
```

```
        this.weatherData = weatherData;
```

```
        weatherData.registerObserver(this);
```

```
    }
```

```
    public void update() {
```

```
        lastPressure = currentPressure;
```

```
        currentPressure = weatherData.getPressure();
```

```
        display();
```

```
    }
```

```
    public void display() {  
        // 在这里显示代码
```

```
    }
```

```
}
```



设计模式 填字游戏答案

